

TP 2 : Filtrage Collaboratif

Le filtrage collaboratif constitue une famille de modèles centraux pour les systèmes de recommandation. Historiquement, ce sont les premiers modèles que l'on a implémentés, que ce soit avec une vision "orienté utilisateurs" ou "orienté articles". Ils ont été améliorés progressivement au cours des années, que ce soit par l'industrie (Amazon et Netflix les ont adoptés très tôt) ou le monde académique. Si des modèles concurrents les complètent aujourd'hui, les modèles de filtrage collaboratif restent encore très communs, comme des briques de bases de systèmes plus complexes.

Dans cette séance, nous allons utiliser un jeu de données appelé `ml-100k`, provenant du groupe de recherche MovieLens (pionniers dans l'étude des systèmes de recommandation). Ces données constituent un jeu standard sur lequel sont évalués de nombreux systèmes de recommandation. Les données "100k" comportent 100 000 évaluations données par des utilisateurs (environ 1000) sur un peu plus de 1600 films. Il existe des versions des données à plus large échelle (1 ou 10 millions de notes). Les notes sont des notes explicites, sur une échelle entre 1 et 5 (meilleure note).

Après avoir pris en main les données, nous allons construire la matrice des interactions (*ratings*), avant d'examiner des manières élémentaires de faire des prédictions, en cherchant les utilisateurs similaires à l'utilisateur courant, puis en pondérant les avis pour prédire. On regardera ces prédictions, avant d'évaluer l'erreur commise. On explorera ensuite l'influence du biais des utilisateurs et on tâchera de limiter la taille du groupe de utilisateurs similaires (paramètre k), avant de tenter de combiner le tout. Enfin, on regardera si des corrélations entre items ressortent.

Le jeu de données MovieLens100k

Le jeu de données s'obtient ici : [MovieLens](<http://files.grouplens.org/datasets/movielens/ml-100k.zip>). Il faut ensuite le dézipper, ce qui devrait créer un dossier `ml-100k`.

Entrons dans le dossier et regardons un peu les données :

```
In [ ]: cd ml-100k/
```

```
In [ ]: ls
```

Vous pouvez lire (au moins partiellement) le README, qui décrit les données.

On s'intéresse surtout au fichier `u.data` qui contient l'entièreté des données, c'est-à-dire 100 000 évaluations. les données comportent 4 colonnes : `userid`, `itemid`, `rating`, `timestamp`.

Regardons les données :

```
In [ ]: !head u.data # 10 premières lignes du fichier
        !echo # line break
        !wc -l u.data # nombre de lignes
```

Chargeons cela dans un dataframe pandas.

```
In [ ]: import numpy as np
        import warnings
        warnings.simplefilter(action='ignore', category=FutureWarning) # éviter des
        Warnings de dépréciation
        import pandas as pd
```

```
In [ ]: names = ['user_id', 'item_id', 'rating', 'timestamp']
        df = pd.read_csv('u.data', sep='\t', names=names)
        df.head()
```

Vérifions que l'on a bien les statistiques annoncées :

```
In [ ]: n_users = df.user_id.unique().shape[0]
        n_items = df.item_id.unique().shape[0]
        print(str(n_users) + ' utilisateurs')
        print(str(n_items) + ' films')
```

Construction de la matrice d'interactions

Nous disposons dans les données MovieLens d'évaluations explicites, nous allons donc proposer un filtrage collaboratif reposant sur ces notes. Nous allons d'abord construire la matrice `ratings` (array NumPy). Comme les données sont indicées à partir de 1 (et que Python indice à partir de 0), un simple décalage suffit.

```
In [ ]: ratings = <votre code>
        ratings
```

Calculons à quel point cette matrice est creuse :

```
In [ ]: sparsity = float(len(ratings.nonzero()[0]))
        sparsity /= (ratings.shape[0] * ratings.shape[1])
```

```
sparsity *= 100
print('Sparsity: {:.2f}%'.format(sparsity))
```

Question : Que pensez-vous de ce chiffre ?

Attention toutefois : ici, on a identifié "valeurs manquantes" et "valeur 0" (hors échelle de notes), en première approximation.

Créons les jeux de données `train` et `test`, en retirant 10 notes par utilisateur, pour les placer dans le jeu de validation (`test`).

```
In [ ]: def train_test_split(ratings):
        test = np.zeros(ratings.shape)
        train = ratings.copy()
        for user in range(ratings.shape[0]):
            test_ratings = np.random.choice(ratings[user, :].nonzero()[0],
                                            size=10,
                                            replace=False)

            train[user, test_ratings] = 0.
            test[user, test_ratings] = ratings[user, test_ratings]

        assert(np.all((train * test) == 0)) # on vérifie que train et test sont
        disjoints
        return train, test
```

```
In [ ]: train, test = train_test_split(ratings)
        train, test
```

Filtrage collaboratif

Similarité Cosinus

Le filtrage collaboratif, qu'il soit orienté utilisateur (*user-based*) ou article (*item-based*), se fonde sur des calculs de similarité. Dans le cas *user based*, on calcule des similarités entre paires d'utilisateurs, en vue d'estimer la proximité de leurs manières de noter.

Une mesure de distance assez commune est la similarité cosinus. Si l'on considère qu'un utilisateur est une ligne (ou une colonne) de la matrice d'interactions, c'est-à-dire un vecteur, la similarité avec un autre utilisateur consiste à calculer l'angle formé par leurs deux vecteurs. Formellement, cela se calcule ainsi :

\$\$
sim(u, u') =

$$\cos(\theta) = \frac{\text{r}_u \cdot \text{r}_u'}{\|\text{r}_u\| \|\text{r}_u'\|} = \frac{\sum_i r_{ui} r_{ui'}}{\sqrt{\sum_i r_{ui}^2} \sqrt{\sum_i r_{ui'}^2}}$$

Les notes étant positives, la similarité cosinus aura une valeur entre 0 (pas de similarité) et 1 (similarité maximale).

On peut écrire une version lente (avec boucles `for`) et une version rapide du calcul, cette dernière reposant sur une vision matricielle et les optimisations des bibliothèques (numpy). La version lente est surtout présentée pour mémoire, elle est *très* lente. On propose une version "user" et une version "item" dans ces fonctions.

```
In [ ]: def slow_user_similarity(ratings, kind='user'):
    if kind == 'user':
        axmax = 0
        axmin = 1
    elif kind == 'item':
        axmax = 1
        axmin = 0
    sim = np.zeros((ratings.shape[axmax], ratings.shape[axmax]))
    for u in range(ratings.shape[axmax]):
        for uprime in range(ratings.shape[axmax]):
            rui_sqrd = 0.
            ruprimei_sqrd = 0.
            for i in range(ratings.shape[axmin]):
                sim[u, uprime] = ratings[u, i] * ratings[uprime, i]
                rui_sqrd += ratings[u, i] ** 2
                ruprimei_sqrd += ratings[uprime, i] ** 2
            sim[u, uprime] /= rui_sqrd * ruprimei_sqrd
    return sim

def fast_similarity(ratings, kind='user', epsilon=1e-9):
    # epsilon -> small number for handling dived-by-zero errors
    if kind == 'user':
        sim = ratings.dot(ratings.T) + epsilon
    elif kind == 'item':
        sim = ratings.T.dot(ratings) + epsilon
    norms = np.array([np.sqrt(np.diagonal(sim))])
    return (sim / norms / norms.T)
```

```
In [ ]: %%timeit slow_user_similarity(train)
```

```
In [ ]: %timeit fast_similarity(train, kind='user')
```

```
In [ ]: user_similarity = fast_similarity(train, kind='user')
item_similarity = fast_similarity(train, kind='item')
print(user_similarity[:5, :5])
```

Question : cherchez l'utilisateur le plus similaire à l'utilisateur 1 (indiqué 0 dans train).

Premières prédictions de notes

Nous allons maintenant pouvoir proposer de premières prédictions de notes pour les évaluations non présentes dans la matrice initiale. On pourra ensuite comparer ces notes avec les données placées dans `test`, pour évaluer la qualité de notre modèle de recommandation.

On prédit une note d'un utilisateur u pour un item i par la moyenne pondérée des notes des autres utilisateurs, avec les poids égaux aux similarités cosinus entre les autres utilisateurs et u , normalisée par le nombre $|r_{\{u\}}|$ de notes :

$$\hat{r}_{\{ui\}} = \frac{\sum_{u'} \text{sim}(u, u') \cdot r_{\{u'i\}}}{\sum_{u'} \text{sim}(u, u')}$$

À nouveau, on utilise les optimisations NumPy pour accélérer les calculs. Le calcul "rapide" n'est, tout de même, pas instantané.

```
In [ ]: def predict_slow_simple(ratings, similarity, kind='user'):
        pred = np.zeros(ratings.shape)
        if kind == 'user':
            for i in range(ratings.shape[0]):
                for j in range(ratings.shape[1]):
                    pred[i, j] = similarity[i, :].dot(ratings[:, j]) \
                                / np.sum(np.abs(similarity[i, :]))
            return pred
        elif kind == 'item':
            for i in range(ratings.shape[0]):
                for j in range(ratings.shape[1]):
                    pred[i, j] = similarity[j, :].dot(ratings[i, :].T) \
                                / np.sum(np.abs(similarity[j, :]))

            return pred

        def predict_fast_simple(ratings, similarity, kind='user'):
            if kind == 'user':
                return similarity.dot(ratings) /
                np.array([np.abs(similarity).sum(axis=1)]).T
            elif kind == 'item':
                return ratings.dot(similarity) /
                np.array([np.abs(similarity).sum(axis=1)])
```

```
In [ ]: %timeit predict_slow_simple(train, user_similarity, kind='user')
```

```
In [ ]: %timeit predict_fast_simple(train, user_similarity, kind='user')
```

Prédictions pour un utilisateur

Conservons la matrice des prédictions :

```
In [ ]: pred = predict_fast_simple(train, user_similarity, kind='user')
```

Question : Calculez les recommandations pour l'utilisateur 1, en proposant un dictionnaire `recodic` contenant les indices des éléments comme clés, et les notes prédites comme valeurs. Éventuellement, proposez un dictionnaire `sortedreco` qui soit une version triée par valeurs de `recodic`.

Indice : vous pouvez utiliser `numpy.partition`.

On peut ensuite essayer de comparer aux notes initiales de l'utilisateur. On se donne tout d'abord une fonction simple permettant de retrouver, à partir de l'identifiant d'un utilisateur (tel que défini dans les données, le premier utilisateur a pour identifiant 1). On se donne ensuite une fonction permettant d'accéder aux titres de films.

```
In [ ]: def note_user_item(user,item):
        return ratings[user-1][item-1]
```

```
In [ ]: colonnes = ['movie id', 'movie title', 'release date', 'video release date', 'IMDb
URL', 'unknown', 'Action', 'Adventure', 'Animation', 'Children', 'Comedy',
'Crime', 'Documentary', 'Drama', 'Fantasy', 'Film-Noir', 'Horror', 'Musical',
'Mystery', 'Romance', 'Sci-Fi', 'Thriller', 'War', 'Western'
]
films = pd.read_csv('u.item', sep='|', names=colonnes, index_col ="movie id")
def titre_film(movieid):
    return films.loc[movieid]["movie title"]

titre_film(49)
```

```
In [ ]: print("Voici le top des éléments pour l'user 0")
for ind,val in sortedreco.items():
    print("Élément %.3i, note prédite : %.2f"%(ind,val))
    print("--          note réelle :",note_user_item(1,ind))
    print(titre_film(ind))
    print()
```

Évaluation : (R)MSE

On peut quantifier l'erreur globale sur le jeu de données, une pratique courante qui permet de comparer des algorithmes de recommandation. On peut utiliser la version MSE (*mean squared error*) ou RMSE (*root mean squared error*).

```
In [ ]: from sklearn.metrics import mean_squared_error
```

```
def get_mse(pred, actual):
    # Keep only nonzero terms.
    pred = pred[actual.nonzero()].flatten()
    actual = actual[actual.nonzero()].flatten()
    # ou squared = False pour la MSE
    return mean_squared_error(pred, actual)
```

```
In [ ]: item_prediction = predict_fast_simple(train, item_similarity, kind='item')
user_prediction = predict_fast_simple(train, user_similarity, kind='user')

print('User-based CF MSE: ' + str(get_mse(user_prediction, test)))
print('Item-based CF MSE: ' + str(get_mse(item_prediction, test)))
```

Question : Que pensez-vous de ce chiffre ?

Influence des k plus proches utilisateurs

Nous avons pour le moment utilisé *tous* les utilisateurs pour calculer les notes. Comme indiqué en cours, il est beaucoup plus fréquent de calculer des groupes de pairs (*peer group*), *i.e.*, des groupes restreints de k utilisateurs très similaires à l'utilisateur pour lequel on veut prédire. L'idée sera ensuite de ne calculer notre somme pondérée qu'avec ces utilisateurs :

$$\hat{r}_{ui} = \frac{\sum_{u' \in \text{limits}_u} \text{sim}(u, u') r_{u'i}}{\sum_{u' \in \text{limits}_u} \text{sim}(u, u')}$$

(le calcul est ici implémenté en version "lente")

```
In [ ]: def predict_topk(ratings, similarity, kind='user', k=40):
    pred = np.zeros(ratings.shape)
    if kind == 'user':
        for i in range(ratings.shape[0]):
            top_k_users = [np.argsort(similarity[:,i])[:-k-1:-1]]
            for j in range(ratings.shape[1]):
                pred[i,j] = similarity[i,:][top_k_users].dot(ratings[:, j]
[top_k_users])
                pred[i,j] /= np.sum(np.abs(similarity[tuple((i, slice(None)))
[top_k_users]))
    if kind == 'item':
        for j in range(ratings.shape[1]):
            top_k_items = [np.argsort(similarity[:,j])[:-k-1:-1]]
            for i in range(ratings.shape[0]):
                pred[i, j] = similarity[j, :][top_k_items].dot(ratings[i, :]
[top_k_items].T)
                pred[i, j] /= np.sum(np.abs(similarity[j, :][top_k_items]))

    return pred
```

```
In [ ]: pred = predict_topk(train, user_similarity, kind='user', k=40)
print('Top-k User-based CF MSE: ' + str(get_mse(pred, test)))
```

```
pred = predict_topk(train, item_similarity, kind='item', k=40)
print('Top-k Item-based CF MSE: ' + str(get_mse(pred, test)))
```

Question : Que pensez-vous de ces chiffres ?

L'idée est évidemment de chercher une valeur de k qui soit la plus satisfaisante possible. Nous allons essayer de visualiser ce qui se passe pour quelques valeurs de k (attention, calcul un peu long).

```
In [ ]: k_array = [5, 15, 30, 50, 100, 200]
user_train_mse = []
user_test_mse = []
item_test_mse = []
item_train_mse = []

def get_mse(pred, actual):
    pred = pred[actual.nonzero()].flatten()
    actual = actual[actual.nonzero()].flatten()
    return mean_squared_error(pred, actual)

for k in k_array:
    print(k)
    user_pred = predict_topk(train, user_similarity, kind='user', k=k)
    item_pred = predict_topk(train, item_similarity, kind='item', k=k)

    user_train_mse += [get_mse(user_pred, train)]
    user_test_mse += [get_mse(user_pred, test)]

    item_train_mse += [get_mse(item_pred, train)]
    item_test_mse += [get_mse(item_pred, test)]
```

```
In [ ]: %matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()

pal = sns.color_palette("Set2", 2)

plt.figure(figsize=(8, 8))
plt.plot(k_array, user_train_mse, c=pal[0], label='User-based train', alpha=0.5,
linewidth=5)
plt.plot(k_array, user_test_mse, c=pal[0], label='User-based test', linewidth=5)
plt.plot(k_array, item_train_mse, c=pal[1], label='Item-based train', alpha=0.5,
linewidth=5)
plt.plot(k_array, item_test_mse, c=pal[1], label='Item-based test', linewidth=5)
plt.legend(loc='best', fontsize=20)
plt.xticks(fontsize=16);
plt.yticks(fontsize=16);
plt.xlabel('k', fontsize=30);
plt.ylabel('MSE', fontsize=30);
```

Question : Que concluez-vous ?

Biais des utilisateurs

Nous pouvons proposer une version débiaisée des notes des utilisateurs. En effet, certains utilisateurs ont une tendance à ne mettre que des bonnes (ou des mauvaises) évaluations. On peut supposer cependant que les différences, pour chaque utilisateur, sont significatives. Si quelqu'un "note bas", ses notes seront (par exemple) entre 1 et 3. On suppose qu'il préfère les films notés 3 aux films notés 1. En revanche, ses 3 ne seront pas comparables à ceux de quelqu'un qui "note haut" (pour lequel les notes seraient entre 3 et 5, 3 étant alors une "mauvaise note").

Pour débiaiser, nous allons retirer des notes de chaque utilisateur la moyenne de ses notes (avant prédiction). On rajoutera cette moyenne à la fin, pour obtenir une prédiction.

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{u'} \text{sim}(u, u') (r_{u'i} - \bar{r}_u)}{\sum_{u'} \text{sim}(u, u')}$$

où $\bar{r}_u$ est la moyenne des notes de u .

```
In [ ]: def predict_nobias(ratings, similarity, kind='user'):
        if kind == 'user':
            user_bias = ratings.mean(axis=1)
            ratings = (ratings - user_bias[:, np.newaxis]).copy()
            pred = similarity.dot(ratings) /
np.array([np.abs(similarity).sum(axis=1)]).T
            pred += user_bias[:, np.newaxis]
        elif kind == 'item':
            item_bias = ratings.mean(axis=0)
            ratings = (ratings - item_bias[np.newaxis, :]).copy()
            pred = ratings.dot(similarity) /
np.array([np.abs(similarity).sum(axis=1)])
            pred += item_bias[np.newaxis, :]

        return pred
```

```
In [ ]: user_pred = predict_nobias(train, user_similarity, kind='user')
        print('Bias-subtracted User-based CF MSE: ' + str(get_mse(user_pred, test)))

        item_pred = predict_nobias(train, item_similarity, kind='item')
        print('Bias-subtracted Item-based CF MSE: ' + str(get_mse(item_pred, test)))
```

Question : Que pensez-vous de ces chiffres ?

Combiner prédictions débiaisées et top- k

Enfin, nous pouvons proposer une version combinée (débiaisée et top-\$\$\$) de notre recommandation simple.

```
In [ ]: def predict_topk_nobias(ratings, similarity, kind='user', k=40):
    pred = np.zeros(ratings.shape)
    if kind == 'user':
        user_bias = ratings.mean(axis=1)
        ratings = (ratings - user_bias[:, np.newaxis]).copy()
        for i in range(ratings.shape[0]):
            top_k_users = [np.argsort(similarity[:,i])[:-k-1:-1]]
            for j in range(ratings.shape[1]):
                pred[i, j] = similarity[i, :][top_k_users].dot(ratings[:, j]
[top_k_users])
            pred[i, j] /= np.sum(np.abs(similarity[i, :][top_k_users]))
        pred += user_bias[:, np.newaxis]
    if kind == 'item':
        item_bias = ratings.mean(axis=0)
        ratings = (ratings - item_bias[np.newaxis, :]).copy()
        for j in range(ratings.shape[1]):
            top_k_items = [np.argsort(similarity[:,j])[:-k-1:-1]]
            for i in range(ratings.shape[0]):
                pred[i, j] = similarity[j, :][top_k_items].dot(ratings[i, :]
[top_k_items].T)
            pred[i, j] /= np.sum(np.abs(similarity[j, :][top_k_items]))
        pred += item_bias[np.newaxis, :]

    return pred
```

```
In [ ]: k_array = [5, 15, 30, 50, 100, 200]
user_train_mse = []
user_test_mse = []
item_test_mse = []
item_train_mse = []

for k in k_array:
    print(k)
    user_pred = predict_topk_nobias(train, user_similarity, kind='user', k=k)
    item_pred = predict_topk_nobias(train, item_similarity, kind='item', k=k)

    user_train_mse += [get_mse(user_pred, train)]
    user_test_mse += [get_mse(user_pred, test)]

    item_train_mse += [get_mse(item_pred, train)]
    item_test_mse += [get_mse(item_pred, test)]
```

```
In [ ]: pal = sns.color_palette("Set2", 2)

plt.figure(figsize=(8, 8))
plt.plot(k_array, user_train_mse, c=pal[0], label='User-based train', alpha=0.5,
linewidth=5)
plt.plot(k_array, user_test_mse, c=pal[0], label='User-based test', linewidth=5)
plt.plot(k_array, item_train_mse, c=pal[1], label='Item-based train', alpha=0.5,
linewidth=5)
plt.plot(k_array, item_test_mse, c=pal[1], label='Item-based test', linewidth=5)
plt.legend(loc='best', fontsize=20)
```

```
plt.xticks(fontsize=16);
plt.yticks(fontsize=16);
plt.xlabel('k', fontsize=30);
plt.ylabel('MSE', fontsize=30);
```

Question : Que concluez-vous ?

Corrélations entre items ?

Comme nous travaillons avec des films, nous pouvons essayer de regarder si des corrélations simples entre les colonnes de la matrices nous paraissent "sensées".

Regardons les données des films :

```
In [ ]: !head -2 u.item
```

Avec les fonctions de similarités construites plus haut, on peut écrire une fonction retournant les films les plus similaires pour un film donné. Le film entré est le premier de la liste (il est le plus similaire à lui-même).

```
In [ ]: idx_to_movie = {}
with open('u.item', 'r', encoding="utf8", errors='ignore') as f:
    for line in f.readlines():
        info = line.split('|')
        idx_to_movie[int(info[0])-1] = info[1]

def top_k_movies(similarity, mapper, movie_idx, k=6):
    return [mapper[x] for x in np.argsort(similarity[movie_idx,:])[:-k-1:-1]]
```

```
In [ ]: idx = 0 # Toy Story
# on utilise ici la fonction item_similarity définie bien plus tôt (retournez la
voir, si nécessaire)
movies = top_k_movies(item_similarity, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

```
In [ ]: idx = 1 # GoldenEye
movies = top_k_movies(item_similarity, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

```
In [ ]: idx = 20 # Muppet Treasure Island
movies = top_k_movies(item_similarity, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

Question : Que pensez-vous de ces listes ?

Pearson, une autre mesure de similarité

```
In [ ]: from sklearn.metrics import pairwise_distances
item_correlation = 1 - pairwise_distances(train.T, metric='correlation')
item_correlation[np.isnan(item_correlation)] = 0.
```

Regardons les mêmes listes :

```
In [ ]: idx = 0 # Toy Story
movies = top_k_movies(item_correlation, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

```
In [ ]: idx = 1 # GoldenEye
movies = top_k_movies(item_correlation, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

```
In [ ]: idx = 20 # Muppet Treasure Island
movies = top_k_movies(item_correlation, idx_to_movie, idx)
for movie in enumerate(movies):
    print(movie)
```

Question : Que concluez-vous ?